Leveraging Model Fusion for Improved License Plate Recognition

What the paper does, what our Kaggle run found, how it fits our research, and what to show and say to the faculty

Prepared for MD. Rokib Hasan Oli (paper reader) · Supervisor: Dr. Md Mansur ul Mulk · 20 October 2026
Paper: R. Laroca, L. A. Zanlorensi, V. Estevam, R. Minetto and D. Menotti (Federal University of Paraná, Brazil), *Iberoamerican Congress on Pattern Recognition (CIARP 2023)*, Springer LNCS, pp. 60-75, DOI 10.1007/978-3-031-49249-5_5. Author version arXiv 2309.04331v2, 15 pages; page numbers below are its pages.
Code: none for the fusion (only the split lists, raysonlaroca.github.io/supp/lpr-model-fusion/).
Our run: Kaggle notebook `paper24_kaggle.ipynb` on Tesla T4 x2, 95 min, all 5 jobs ended with `code=0`. Outputs: `After Train/results.zip`. Every number of ours below was recomputed from the per-plate predictions in that zip (`report_src/analysis.py`).

The whole report on one page

The paper in three lines. Read the same plate crop with several different OCR models and combine their answers. The best rule is a plain majority vote over whole strings (ties broken by confidence or by the best model). With 12 models on 8 datasets the mean plate accuracy rises from 92.4 % (best single model) to 97.6 % (p.9), and on 4 unseen datasets from 87.6 % to 90.3 % (p.11). Trusting the most confident model is the worst rule. Fusing 4-6 fast models gives most of the gain for a fraction of the time (p.12).

What our run found. (1) **The paper's recipe gives almost nothing on our data:** fusing our 4 Bangla models lifts plate accuracy from 85.56 % to 86.18 % (+0.62 points: 7 plates fixed, 3 broken, not significant, p = 0.34). The reason is measurable: two of the four models are weak, and the models make the same mistakes. On 77 of the 94 plates our best model reads wrongly, no other model is right either. (2) **The paper's warning about confidence holds:** CRNN gives 60 % of its wrong readings a confidence of 0.9 or more, so "take the most confident" drops accuracy to about 76 %. (3) **Our Bangla-specific change works:** the BRTA plate layout (district, optional Metro, one letter, six digits) is a free error detector: on our test set every reading that broke it was wrong. Asking CRNN only for those 9.8 % of plates gives **87.40 % (+1.84 points, 12 fixed, 0 broken, p = 0.0005)** at 12.2 ms per plate instead of 32.5 ms. (4) We found a dataset bug: 19 of the 30 BLPD "plate crops" in the OCR test set are whole vehicle photos. On real crops only, the same method reads 89.9 % of plates. Both follow-ups are already done: the bug is fixed in dataset v1.1, and the new rule is in the team kit (section 18).

Is the run OK? Yes for the question it asks. All jobs finished inside their caps, and our recomputation matches the Kaggle table exactly (86.18 % and oracle 88.17 %). ViTSTR (23.20 %) and the slot CNN (45.93 %) are undertrained, so they are weak voters (section 11).

Why it matters for us. Our proposal already cites this paper for multi-frame voting (RQ3, O4) and promises BRTA-format-aware correction (RQ2, O3). This run turns both into numbers: majority voting needs strong and different voters, and the plate grammar is the cheapest, safest fusion signal we have.

One sentence for the faculty. "We tested the paper's model-fusion idea on Bangla plates. Plain voting barely helps because our models make the same mistakes. Using the BRTA plate format to decide when to ask a second model gives a significant +1.8 points almost for free, and the same rule will drive our video voting."

Layout. Part A (sections 1-7): understand the paper. Part B (8-12): our experiment; read it before you quote our numbers. Part C (13-19): research use, benefits, slides, script, questions, next steps.

1. What problem does the paper solve?

A plate reader has two stages: find the plate (detection) and read its characters (recognition, OCR). Detection is nearly solved by YOLO-type models, so recent work focuses on recognition (p.1-2). But earlier studies, several by the same group, showed that recognition models are **uneven**: a model that is best on one dataset can be poor on another, because each dataset has its own layouts, fonts, tilt and image quality (p.2).

So the paper asks three plain questions (p.2):

1. Can we get clearly better plate reading by **combining the outputs** of several different recognition models?
2. If yes, **how much** better?
3. **How many and which** models should we combine?

Before this paper, LPR fusion work had used copies of one model (Izidio et al., several Tiny-YOLOv3 nets on a private set) or synthetic degraded images (Schirrmacher et al.); nobody had combined many *different* models across many public datasets (p.4). The authors also stress a practical point: some applications (traffic tickets, forensics) prefer accuracy over speed, so they can afford several models (p.2).

2. Key terms in plain words

Term	Meaning
Model fusion / ensemble	Run several models on the same input and combine their answers. It helps only when the models make *different* mistakes.
Recognition rate (plate exact)	Share of plates whose **whole** string is right. One wrong character = wrong plate (p.8). This is the paper's only metric.
HC (highest confidence)	Pick the string of the model that is most sure of itself, even if no other model agrees (p.7).
MV (majority vote)	Pick the full string that most models output. Confidence is ignored unless there is a tie (p.7).
MVCP (vote per character position)	Vote separately on character 1, character 2, ... and join the winners. Breaks when one model drops or adds a character, because the positions shift (p.10-11).
-HC / -BM tie-break	If two strings get the same number of votes: take the more confident one (-HC) or the one from the "best model" (-BM) (p.7-8).
Top-N	Fuse the N best models (ranked by mean accuracy). Table 4 tries N = 2 ... 12.
Intra-dataset / cross-dataset	Test on the test split of a dataset used in training / on a dataset never seen in training (harder, closer to a new camera).
Oracle (our term)	A plate counts as right if *any* model is right. No voting rule can beat it, so it is the ceiling for fusion.
Calibration (our term)	A model is calibrated if, among readings with confidence 0.9, about 90 % are right. Uncalibrated confidences make HC fail.

3. The 12 models and the 12 datasets

The models were chosen because they cover the design patterns of modern text recognisers and are common baselines in LPR (p.5). Each was trained **once** on the union of 8 training sets, in its own original framework:

Family	Models	How they read	Framework, training (p.5)
Scene-text, CTC	CRNN, GRCNN, Rosetta, STAR-Net	CNN features, (BiLSTM), then CTC decoding	PyTorch (Clova AI fork), Adadelta, 300 k iterations, batch 128
Scene-text, attention	R2AM, RARE, TRBA	CNN, (TPS rectification), BiLSTM, attention decoder	same
Transformer	ViTSTR-Base	ViT patches; output tokens are the characters	same
Multi-task CNN	Holistic-CNN, Multi-Task-LR	CNN, then one classifier per character slot	Keras, Adam 1e-3, batch 64, early stop
YOLO character detectors	CR-NET, Fast-OCR	detect every character as an object, sort the boxes	Darknet, SGD, 90 k iterations, batch 64

Data (p.5-7, Table 2). 8 datasets for training and testing: Caltech Cars, EnglishLP, UCSD-Stills, ChineseLP, AOLP, SSIG-SegPlate, UFPR-ALPR, RodoSol-ALPR (20,000 images; Brazilian and Mercosur plates). 4 datasets

only for cross-dataset tests: OpenALPR-EU, PKU, CD-HARD (heavily tilted, mixed layouts) and CLPD. Training also used 772 web images and synthetic plates made by **shuffling character positions**, so that models cannot memorise sequences (p.6-7). Crops come from a YOLOv4 detector, not from the ground truth, to be realistic (p.8).

> **Mapping to our run.** We trained one model from each of four of these families on our data: char-YOLOv8n (the CR-NET / Fast-OCR family), CRNN-CTC (CRNN), a slot CNN in the fast-plate-ocr style (Holistic-CNN / Multi-Task-LR) and ViTSTR-tiny (ViTSTR). Section 8 gives the details.

4. The method: three rules and two tie-breaks

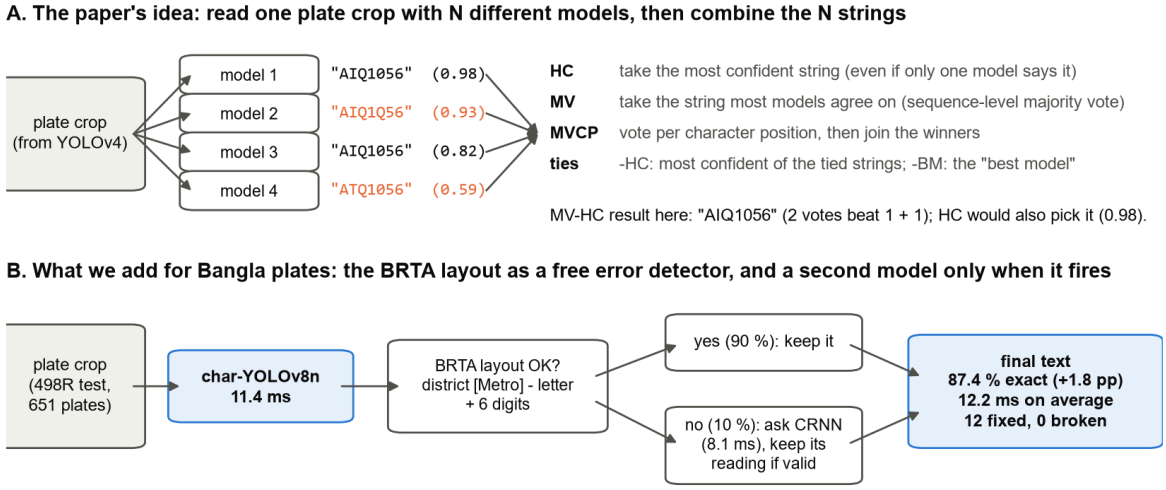


Figure 1. A: the paper's fusion (example strings from its Fig. 4, p.10; orange = wrong). B: the change we test on Bangla plates (section 10, finding 4). Numbers in B are ours, on the 498R test split.

The whole method fits in a few lines. For each plate, every model gives a string and a confidence. Then:

- **HC:** output the string with the highest confidence (p.7).
- **MV:** group identical strings and output the largest group. In the paper's example, five models say "ABC-123", "ABC-123", "ABC-124", "ABC-124", "ABC-125": a tie between two strings (p.7).
- **MVCP:** vote on every character position and join the winners (p.7).
- **Ties:** -HC takes the tied string with the higher confidence; -BM takes the tied string that the best single model output (p.7-8). With only **two** models every disagreement is a tie, so MV-BM is just "trust the best model" and MV-HC is just HC. You need at least three models before voting does anything.
- **Confidences are used raw.** The authors tried normalising them per model; it did not help (p.8).

The "best model" for -BM and the top-N order are taken from the mean accuracy over the datasets. The authors say the validation ranking is "essentially the same, with only two models swapping positions" (p.9).

5. Every result in the paper

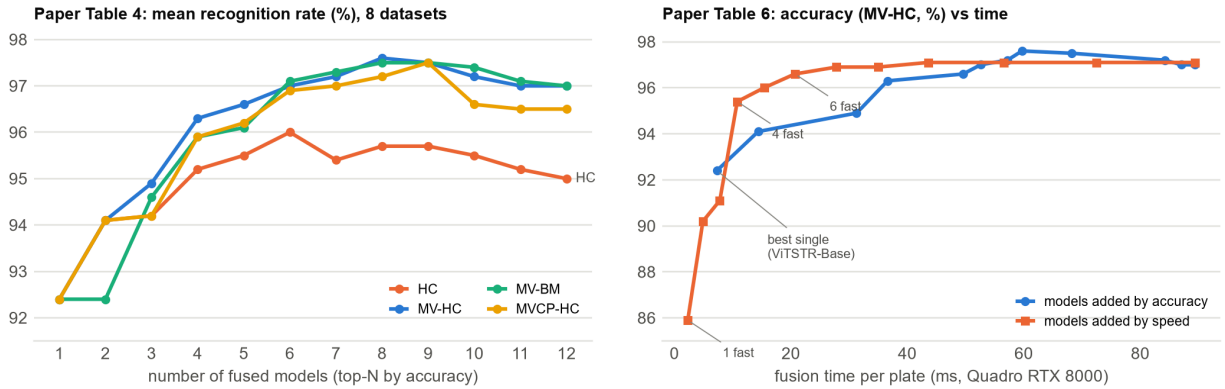


Figure 2. The paper's numbers, redrawn. Left: Table 4 (p.9), fusing the top-N models. Right: Table 6 (p.12), accuracy vs time, models added by accuracy or by speed.

Result	Value	Page
Best single model, 8 datasets (mean)	ViTSTR-Base 92.4 %; STAR-Net 92.2; TRBA 91.1; CR-NET 90.8; ... CRNN 87.1; Multi-Task-LR 85.9	p.9, Table 3
Best fusion, 8 datasets	**MV-HC (top 8) 97.6 %**; MV-BM (top 8) 97.5; MVCP-HC (top 9) 97.5; MVCP-BM 97.2; HC (top 6) 96.0	p.9, Table 3
Worst cases removed	every single model is below 90 % on at least two datasets; every fusion rule is above 90 % on all eight	p.9
Top-N curve (MV-HC)	top 2: 94.1, top 3: 94.9, top 4: 96.3, top 6: 97.0, top 8: 97.6, top 12: 97.0 %	p.9, Table 4
Cross-dataset, best single	Fast-OCR 87.6 %; CR-NET 87.1; STAR-Net 84.4; ViTSTR-Base only 75.9	p.11, Table 5
Cross-dataset, best fusion	**MV-BM (top 8) 90.3 %**; MV-HC 90.1; MVCP-HC 87.6; HC 84.4 (below the best single model)	p.11, Table 5
Hardest unseen set (CD-HARD)	best single 59.6 → MV fusion 65.4 %	p.11, Table 5
Speed, single models	Multi-Task-LR 2.3 ms; Holistic-CNN 2.5; CRNN 2.9; Fast-OCR 3.0; ViTSTR-Base 7.3; TRBA 16.9 ms per plate	p.12, Table 6
Speed, fusion	top 8 by accuracy: 59.7 ms (17 FPS). The 4 / 5 / 6 fastest: 95.4 / 96.0 / 96.6 % at 10.8 / 15.4 / 20.7 ms	p.12, Table 6
Surprise	the 3 fastest fused (91.1 %) are below the best single model (92.4 %)	p.12
Training time, seeds, significance tests	not reported	-

The authors' own explanation (p.10): fusion "does not necessarily lead to significantly improved performance compared to the best model in the ensemble. Instead, it reduces the likelihood of obtaining poor performance", because diverse models make different errors but agree when they are right. HC is weak because every model is also confidently wrong at times (p.11, Fig. 4). Keep this sentence in mind: our run shows exactly when that condition fails.

6. Critical review

Strengths

- **Wide and honest test bed:** 12 models in their original frameworks, 12 public datasets, intra- and cross-dataset results, and detector crops instead of perfect crops (p.5-8).
- **Simple, reusable rules:** no training, about 40 lines of code; anyone can add it to an existing system.
- **Speed is reported next to accuracy** (Table 6), which gives a practical recipe: 4-6 fast models.
- **Useful negative results:** HC is the worst rule and confidence normalisation did not help (p.8, p.11).

Weaknesses (what a reviewer would ask)

- **Test-informed choices.** The "best model" for -BM and the top-N order come from test-set accuracy across datasets. The authors say validation gives nearly the same order (p.9), but strictly every choice should be made on validation only. *We did that in our run.*
- **One training run per model, no confidence intervals, no significance test.** Several gains in Table 4 are differences of 0.1-0.3 points on small test sets (Caltech Cars has 46 plates, UCSD-Stills 60).
- **The headline gain partly measures "not knowing which model to use".** We recomputed it from Table 3: if you could pick the best single model *for each dataset*, the mean would already be **96.8 %**. Fusion (97.6 %) adds only **0.8 points** over that. The big +5.2 is against one fixed model (ViTSTR-Base), which is weak on the older US and EU sets. Cross-dataset the per-dataset best is Fast-OCR everywhere (87.6 %), so the +2.7 there is a real gain, mostly on CD-HARD and CLPD.
- **Heavy for real time:** 8 models cost 59.7 ms per plate on a 48 GB workstation GPU; much slower on a laptop CPU.
- **Latin and Chinese plates only**; no complex script such as Bangla, where one character can be several Unicode code points.
- **Diversity is assumed, not measured.** The paper never reports how often the models fail together, which is exactly what decides whether fusion can work (our finding 2).

Verdict. A careful, peer-reviewed and easy-to-copy study. Its main lesson is about *risk reduction* across datasets rather than a large gain on one dataset, and its claim depends on having many strong and different models. Both points matter for us.

7. What would break on our Bangla plates (and what we did)

Problem	Why	What we did / do
MVCP on Unicode	Bangla vowel signs and conjuncts change the string length, so character positions shift (worse than in the paper)	vote on **tokens** (district, মেট্রো, letter, 6 digits) instead of Unicode characters
Correlated models	all our models learn from the same 5,607 crops; 2,136 of their labels came from a character YOLO (auto-labels), so they can share its mistakes	measure it: pairwise overlap and the oracle (finding 2)
Few strong models	we have one strong reader (char-YOLO); training several strong Bangla OCR models is expensive	test fusion with 2-4 models; use teammates' models later (section 13)
Confidence scales differ	a box score (char-YOLO) and a softmax mean (CRNN) do not mean the same thing	check calibration (finding 3); prefer MV-BM over HC
Real time (≥ 10 FPS, RQ3)	running 4 models on every plate adds up	call a second model only when needed (finding 4)
Two-row plates	most Bangla plates have two rows	char-YOLO reads boxes by token kind; CRNN/ViTSTR see the two rows side by side

PART B · OUR EXPERIMENT

8. What we ran and why

No fusion code and most of the paper's datasets are not public (several need a request), so a run on the paper's own data was not possible. Instead the kit asks the paper's question on **our** data: does fusing different OCR models help on Bangladeshi plates, and at what cost?

Run	Where	Model and settings	Train time
char-YOLOv8n	GPU 0	YOLOv8n from COCO weights; 100 epochs, 320 px, batch 32, SGD; 104 token classes	62 min
slot CNN	GPU 0 (after)	6-block CNN, 9 slots, 64×128 grey input; 100 epochs, AdamW + OneCycle; 1.73 M parameters	29 min
CRNN-CTC	GPU 1	VGG-style CNN + 2-layer BiLSTM + CTC, 48×256; 60 epochs; 7.69 M parameters	28 min
ViTSTR-tiny	GPU 1 (after)	ImageNet ViT-tiny (timm), 32×256, first 12 tokens classified; 60 epochs, lr 3e-4; 5.41 M	29 min
fusion	CPU	every subset of 2-4 models × HC, MV-HC, MV-BM, MVCP-HC; best chosen on validation	0.3 min

Item	Setting
Data	498R BanglaLPR All-in-One v1.0, OCR split: 5,607 train crops (3,471 human + 2,136 auto-labelled), 766 validation, 651 test (human labels only). Sources in test: LPDBA 202, ANPR 184, KUET 183, NightLPBD 52, BLPD 30.
Split	grouped 70/15/15, seed 42, no near-duplicate across splits (team rule)
Metric	plate exact (whole plate right after Unicode normalisation, digits as Bangla, no spaces or hyphens), plus CER and character accuracy
Fair choice	MV-BM's "best model" and the best fusion are chosen on **validation**; test is read once. (The paper chose on test.)
Speed	ms per plate, batch 1, on one T4; a fusion costs the sum of its members (run one after another)
Hardware, software	Kaggle Tesla T4 x2, one job per GPU; torch 2.11 + CUDA 12.8, Ultralytics 8.4.170, timm 1.0.29

After the run we went further than the notebook (all in `report_src/analysis.py`, no retraining): error overlap between models, a significance test, confidence calibration, three extra fusion rules, a speed-aware fallback, and results per condition and per source. Every extra rule was again chosen on validation and reported on test.

9. Results next to the paper

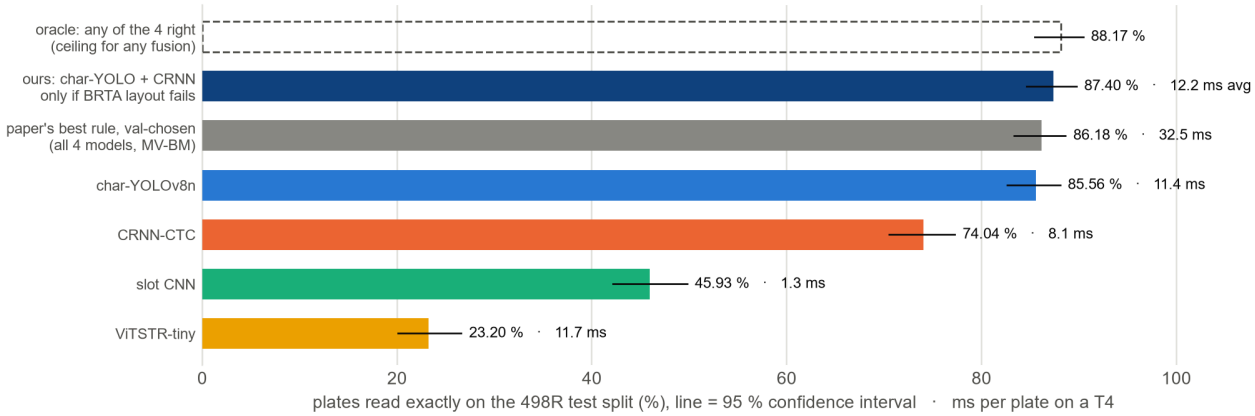


Figure 3. Our test results (651 plates). The black lines are 95 % confidence intervals (Wilson): about ±2.7 points for the best model, so a gain of 0.6 points cannot be told apart from chance.

Metric	Paper (page)	Ours (498R test)
Best single model, plate exact	92.4 % ViTSTR-Base, mean of 8 sets (p.9)	**85.56 %** char-YOLOv8n (val 81.07 %)

Other single models	85.9-92.2 % (all 12 within 6.5 points)	CRNN 74.04, slot CNN 45.93, ViTSTR-tiny 23.20 % (spread 62 points)
Best fusion with the paper's rules	97.6 % MV-HC top 8 (+5.2); cross 90.3 % (+2.7)	86.18 %, all 4 models, MV-BM (+0.62; p = 0.34)
HC (most confident)	96.0 % intra; 84.4 % cross, below the best single model	76.0-76.5 % for every subset with char-YOLO and CRNN, far below the best single model
Oracle (any model right)	not reported	88.17 % (ceiling: only +2.61 points)
Fusion time per plate	59.7 ms top 8 (RTX 8000); 10.8 ms for the 4 fastest	32.5 ms for all 4 (T4)
Our format-aware fallback	-	**87.40 %** (+1.84; 12 fixed, 0 broken; p = 0.0005) at 12.2 ms
Training time	not reported	148 GPU-min in total (95 min wall time on 2 GPUs)

> **Read the gap as a direction, not a contest.** The paper used 12 other datasets with Latin and Chinese plates, larger training sets and 12 tuned models. Our numbers are on Bangla plates, with 4 models of which two are undertrained. What carries over is the *pattern*: which rule wins, and when fusion can help at all.

Top-N by validation accuracy (paper Table 4 style)	HC	MV-HC	MV-BM	ms
top 1 (+ char-YOLOv8n)	85.56	85.56	85.56	11.4
top 2 (+ CRNN-CTC)	76.19	76.19	85.56	19.5
top 3 (+ slot CNN)	76.50	78.19	85.87	20.8
top 4 (+ ViTSTR-tiny)	76.34	78.19	86.18	32.5

The table shows the paper's own pattern on our data: HC and MV-HC fall as soon as CRNN joins, and MV-BM creeps up by 0.3 points per extra model. With two models MV-BM equals the best model alone, as explained in section 4.

10. Six findings

Finding 1: the paper's recipe is not significant on our data

The best fusion chosen on validation is all four models with MV-BM: 81.85 % on validation and 86.18 % on test, against 85.56 % for char-YOLOv8n alone. Per plate, fusion fixes 7 readings and breaks 3, a net of 4 plates out of 651. An exact McNemar test (the right test for two readers on the same plates) gives p = 0.34: no evidence of a real gain. It also triples the time (11.4 → 32.5 ms).

Finding 2: our models fail together, so voting has little to work with

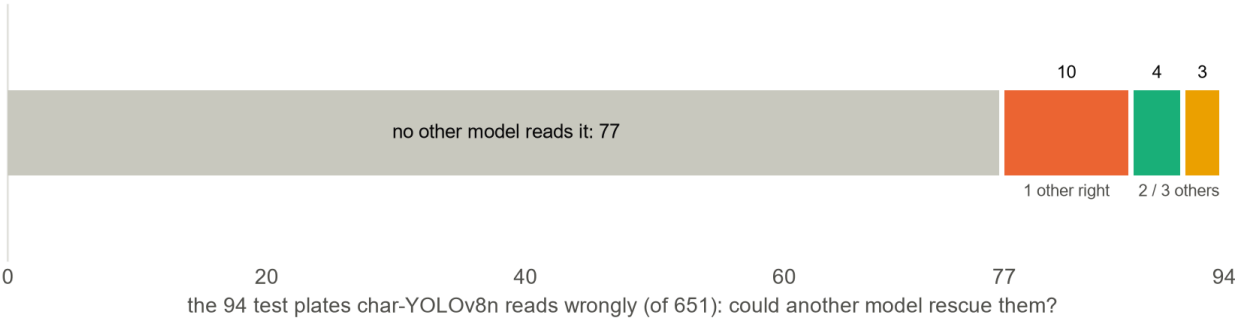


Figure 4. Of the 94 plates char-YOLOv8n reads wrongly, 77 are read wrongly by all four models. Only 17 could be rescued by any rule, so the oracle is 88.17 %.

- When char-YOLO is wrong, CRNN is also wrong 85 % of the time (Yule's Q = 0.94; 0 would mean independent errors, 1 fully shared).
- If three readers each as good as char-YOLO made *independent* errors, a majority vote would be right 94.3 % of the time. Real errors are shared, which is why we get 86.2 %.
- For a wrong plate to be fixed by voting, at least two other models must agree on the right string. That happened on only 7 plates.
- The paper's 12 models were all strong (85.9-92.4 %), so a wrong model was usually outvoted. Here ViTSTR-tiny and the slot CNN are wrong on most plates and cannot outvote anything.

Finding 3: same confidence, different meaning (why HC fails)

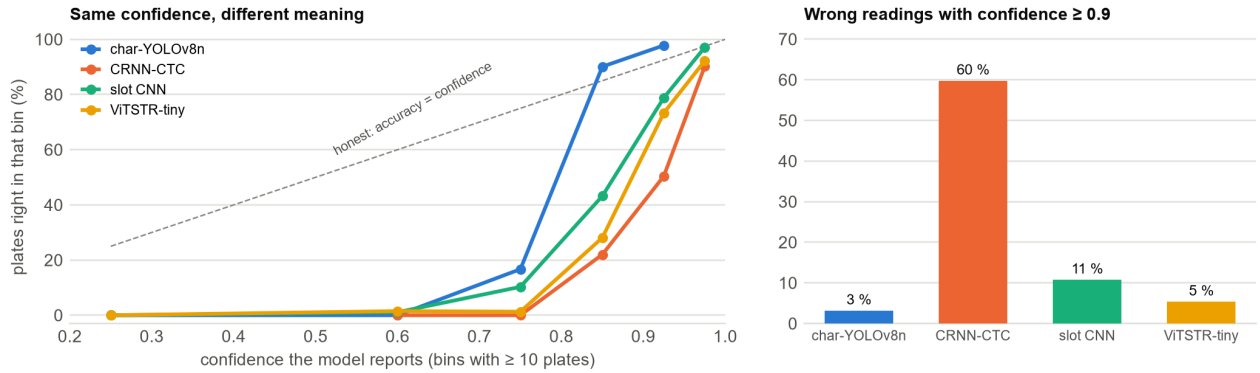


Figure 5. Left: share of right readings in each confidence bin. Char-YOLO's scores are honest above 0.8; CRNN's are not (only 50 % right between 0.9 and 0.95). Right: share of wrong readings with confidence ≥ 0.9.

60 % of CRNN's wrong readings have confidence ≥ 0.9 (mean 0.88 when wrong, 0.98 when right), against 3 % for char-YOLO. So when the two disagree, HC usually believes CRNN and loses: HC with char-YOLO + CRNN reads 76.2 %, 9 points *below* char-YOLO alone. In the char-YOLO + ViTSTR pair, HC overrules 28 right readings and fixes only 5. This is the paper's own finding (p.11), only stronger: within one model the confidence ranks readings well (AUROC 0.86-0.95), but across models the scales do not match.

Finding 4 (our contribution): the BRTA layout is a free and safe fusion signal

Every Bangladeshi plate has one layout: a district, an optional মেট্রো, exactly one letter, then six digits. A reading that breaks it cannot be right. We used this in two ways, both chosen on validation:

- **As an error detector.** 64 of char-YOLO's 651 test readings (9.8 %) break the layout, and **all 64 are wrong** (precision 100 %). That is 68 % of all its errors, found without any second model. The readings that pass are 94.9 % right.
- **As a fallback rule.** Keep char-YOLO's reading when the layout is valid; otherwise ask CRNN and take its reading if that one is valid. CRNN runs on only 9.8 % of plates, so the mean cost is 12.2 ms instead of 19.5 ms. Result: **87.40 %**, 12 plates fixed and **none broken** (McNemar p = 0.0005). Because it only replaces readings that are certainly wrong, it cannot break a right one.
- The same filter inside the voting (MV-BM+format) gives the same 87.40 % with char-YOLO + CRNN, and helps every subset (Figure 6). Also asking CRNN when char-YOLO's confidence is low (thresholds 0.5-0.95) did not change the validation score, so the rule uses the layout alone.

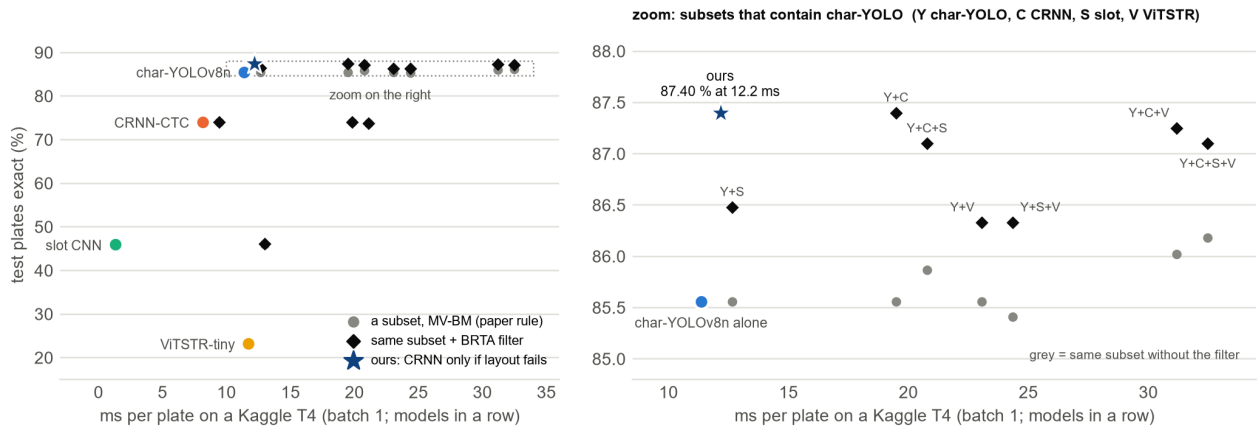
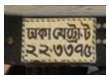


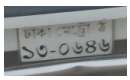
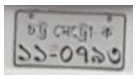
Figure 6. Accuracy vs time for every subset (paper Table 6 style). Grey: the paper's MV-BM rule; black diamonds: the same subset with the BRTA filter; star: our fallback, which runs CRNN only when the layout check fails. The filter lifts every subset; the fallback is the cheapest point on the top edge.

Why char-YOLO breaks the layout (64 plates; a plate can have several reasons): no letter found 26, no district 24, more than six digits 22, fewer than six 20, two letters 10 (one letter detected twice with two classes, e.g. গখ for গ), two districts 3. The other 30 wrong readings have a valid layout with one wrong symbol; no rule can catch those without a second opinion. Most errors are **missed** characters, not confused ones, so the next fix belongs in the decoder (section 18).

Examples (test crops; red = wrong; small number = confidence)

Crop	Truth	char-YOLOv8n	CRNN	Our fallback
	ঢাকা মেট্রো-গ ২৬-৭৬৬৩	ঢাকা মেট্রো-গখ ২৬-৭৬৬৩ 0.80	ঢাকা মেট্রো-গ ২৬-৭৬৬৩ 0.98	ঢাকা মেট্রো-গ ২৬-৭৬৬৩
	যশোর-ট ১১-৪২৬৭	১১-৪২৬৭ 0.77	যশোর-ট ১১-৪২৬৭ 0.98	যশোর-ট ১১-৪২৬৭
	চট্ট মেট্রো-ম ৫১-১০৩২	চট্ট মেট্রো-ম ৩৫১১০৩২ 0.84	চট্ট মেট্রো-ম ৫১-১০৩২ 1.00	চট্ট মেট্রো-ম ৫১-১০৩২
	খুলনা মেট্রো-ট ১১-০৮৯৯	খুলনা মেট্রো-ট ১১০০৮১৯২ 0.74	খুলনা মেট্রো-ট ১১-০৮৯৯ 0.97	খুলনা মেট্রো-ট ১১-০৮৯৯
	ঢাকা মেট্রো-ট ২২-৩৩৭৫	ঢাকা মেট্রো-হট ২২-৩৩৭৫ 0.81	ঢাকা মেট্রো-ট ২২-৩৩৭৫ 0.99	ঢাকা মেট্রো-ট ২২-৩৩৭৫

Above: plates the fallback fixes. Char-YOLO finds a letter twice (গখ), misses the district at night, or finds seven or eight digits; each breaks the layout, so CRNN is asked and its valid reading is kept. Below: all three plates the paper's 4-model vote *breaks*. CRNN is confidently wrong about the letter and the slot CNN makes the same mistake, so two votes beat char-YOLO's right answer. CRNN and the slot CNN confuse the same letter pairs (ঠ/ব, খ/ক, ক/গ): shared errors again.

Crop	Truth	char-YOLOv8n	CRNN	slot CNN	ViTSTR	4-model MV-BM
	ঢাকা মেট্রো-ঠ ১৩-০৬৪৬	ঢাকা মেট্রো-ঠ ১৩-০৬৪৬ 0.84	ঢাকা মেট্রো-ব ১৩-০৬৪৬ 0.94	ঢাকা মেট্রো-ব ১৩-০৬৪৬ 0.90	ঢাকা মেট্রো-ব ১৩-০৬৪৪ 0.87	ঢাকা মেট্রো-ব ১৩-০৬৪৬
	চট্ট মেট্রো-খ ১১-০৭৩০	চট্ট মেট্রো-খ ১১-০৭৩০ 0.89	চট্ট মেট্রো-ক ১১-০৭৩০ 0.99	চট্ট মেট্রো-ক ১১-০৭৩০ 0.88	ঢাকা মেট্রো-ক ১১-০৭৭০ 0.85	চট্ট মেট্রো-ক ১১-০৭৩০
	চট্ট মেট্রো-ক ১১-০৭৯৩	চট্ট মেট্রো-ক ১১-০৭৯৩ 0.82	চট্ট মেট্রো-গ ১১-০৭৯৩ 0.98	চট্ট মেট্রো-গ ১১-০৭৯৩ 0.88	চট্ট মেট্রো-গ ১১-০৭৯৩ 0.92	চট্ট মেট্রো-গ ১১-০৭৯৩

Finding 5: a dataset bug: most BLPD "plate crops" are whole photos



Figure 7. BLPD images in the OCR test split. Top: 6 of the 19 that are whole vehicle photos (640×640). Bottom: real plate crops, also stored at 640×640.

Every model scores ≤ 37 % on BLPD, so we looked at the images. 19 of the 30 BLPD test "crops" show the whole vehicle, with the plate a few percent of the image; an OCR model cannot read that, and the 4 models together read 1 of them. On the 11 real crops char-YOLO reads 91 %. The same problem hit training and validation: 63 of the 130 BLPD training rows and 9 of the 25 validation rows, in both `ocr/` and `recognition/`. Without the 19 photos (632 real crops), char-YOLO reads **88.0 %**, paper fusion 88.6 %, our fallback **89.9 %** and the oracle 90.7 %.

> **Fixed on 20 Oct (dataset v1.1).** A new build step cuts all 91 whole photos to their plates using the human character boxes. Splits and counts are unchanged, and the verify script reports no problems. Still to do: upload the v1.1 zip, set its Drive id, re-run. The numbers in this report are from v1.0.

Finding 6: where it helps (conditions and sources)

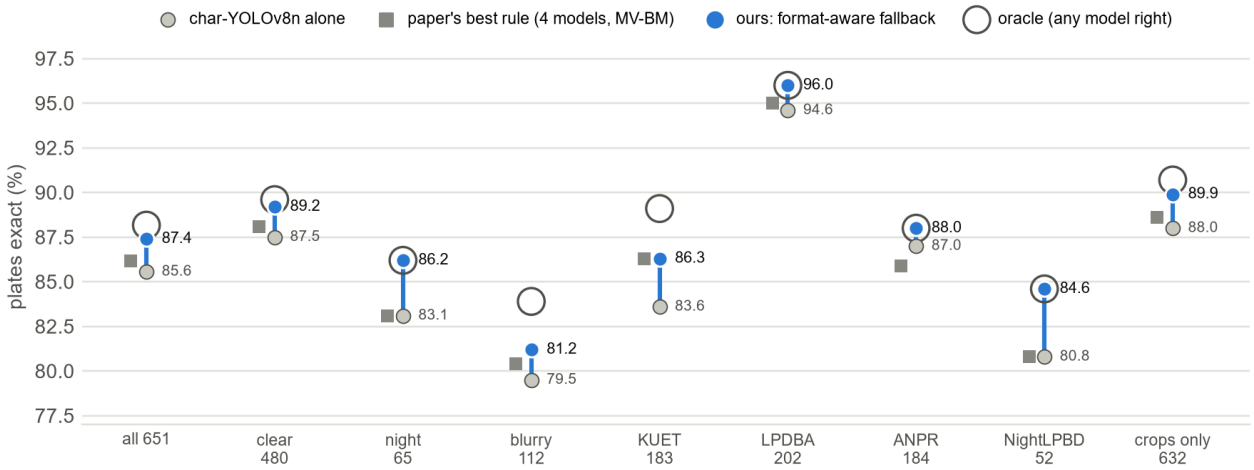


Figure 8. Plate exact by condition and source. Grey dot: char-YOLO alone; square: the paper's 4-model vote; blue dot: our fallback; ring: oracle. Where the blue dot sits in the ring, our rule already reaches what any fusion of these four models could reach.

Group (plates)	char-YOLO	CRNN	slot	ViTSTR	paper fusion	our fallback	oracle
clear (480)	87.5	78.1	48.8	26.9	88.1	**89.2**	89.6
night (65)	83.1	67.7	20.0	4.6	83.1	**86.2**	86.2
blurry (112)	79.5	60.7	49.1	17.0	80.4	**81.2**	83.9
KUET (183)	83.6	75.4	59.6	43.7	86.3	**86.3**	89.1
LPDBA (202)	94.6	85.1	55.4	24.8	95.0	**96.0**	96.0
ANPR (184)	87.0	69.6	35.9	9.8	85.9	**88.0**	88.0
NightLPBD (52)	80.8	67.3	15.4	5.8	80.8	**84.6**	84.6
BLPD (30)	36.7	30.0	13.3	0.0	36.7	**36.7**	36.7
real crops only (632)	88.0	76.3	47.3	23.9	88.6	**89.9**	90.7

The fallback helps most where char-YOLO *misses* characters: night (+3.1 points) and NightLPBD (+3.8), where missed district names are common. The paper's vote adds nothing at night, because the weak models are weakest there (slot CNN 20 %, ViTSTR 5 %). On blurry plates and on KUET the oracle is still 2-3 points higher, so a stronger second reader would help there. Weather tests from the notebook (char-YOLO alone): real fog copies 73.1 % (26 plates), synthetic rain 70.5 %: the biggest drops of all.

11. Health check: is the run OK?

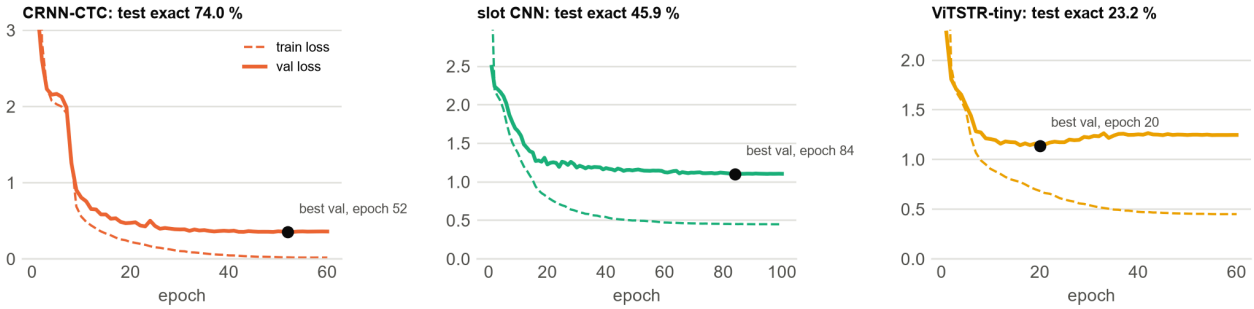


Figure 9. Training (dashed) and validation (solid) loss of the three sequence models. The black dot is the epoch whose weights were kept.

Check	Result	Meaning
Jobs and time caps	all 5 jobs code=0; char-YOLO 100/100 epochs; 95 min wall time	nothing was cut short
Our numbers = Kaggle's	recomputed best fusion 86.18 = 86.18; oracle 88.17 = 88.17	the analysis reads the same predictions

No test peeking	best model, tie-break model and every rule chosen on validation	test numbers are fair
char-YOLOv8n	box mAP50 0.913 on test; character accuracy 95.9 %, CER 4.07 %	healthy; errors are mostly missed boxes (recall 0.89)
CRNN-CTC	val loss flat from epoch ~40, best at 52; test 74.04 %	healthy but weaker than char-YOLO
ViTSTR-tiny	val loss lowest at epoch 20, then rises; test 23.20 %	**overfits and undertrained:** about 5,200 steps vs 300 k iterations in the paper; tiny model, 32-px input
slot CNN	train loss at the label-smoothing floor, val loss 1.10; test 45.93 %	**underfits the task:** 1.7 M parameters from scratch, global max-pool loses position
Validation vs test	every model is 2-10 points better on test than on validation (char-YOLO 81.1 vs 85.56)	the validation plates are harder for every model; the ranking is the same on both, so the choices still hold
Auto-labels in training	2136 of 5607 OCR training labels are auto-labels from a character YOLO	a likely reason why the models share errors (finding 2)
BLPD whole photos	19 of 30 BLPD test images are not crops (v1.0)	costs char-YOLO 2.4 points (the others 0.7-2.3); fixed in dataset v1.1
Single run	one seed per model, as in the paper	add 3 seeds for char-YOLO and CRNN before final claims

12. Ours vs the proposal's targets

Proposal target	Where we are (this run)	Status
Character recognition accuracy ≥ 90 % on test crops (RQ2)	char-YOLOv8n 95.9 %; with the fallback 95.9 %	met at character level
Plate read exactly (our stricter view)	85.56 % → 87.40 % with the fallback; 89.9 % on real crops	close to 90 %; next fixes in section 18
≥ 10 FPS full pipeline (RQ3, O4)	OCR 12.2 ms per plate on a T4 (≈ 82 plates/s); CPU not yet measured	OCR part is not the bottleneck
BRTA-format-aware correction (O3)	layout check finds 68 % of errors with 100 % precision	first working piece
Multi-frame voting (RQ3, O4)	not tested here (images only)	this paper's rule + our filter, next

13. How to use this paper in our research

Use	What to do	Links to
1. Final OCR choice	Use char-YOLO as the main reader and CRNN as a fallback when the BRTA layout fails. Report it as plate exact vs ms per plate, like the paper's Table 6 (our Figure 6).	O3, RQ2
2. Grammar-aware fusion	Make the BRTA layout part of every fusion rule (MV-BM+format). It is the Bangla-specific part the paper does not have, and it never hurts a right reading.	O3 (format-aware correction)
3. Multi-frame voting in video	The same rule, but the "models" are the frames of one tracked vehicle: drop invalid readings, take the majority string, stop OCR once 3 valid readings agree. The proposal already cites this paper for it.	RQ3, O4
4. A team ensemble for free	p24_fusion.py accepts any run folder with ocr_preds_val.csv and ocr_preds_test.csv on the same split. Since 20 Oct, the PARSeq (#31), VLM (#22) and ved.py scripts also write the validation file, so teammates' readers can join without retraining.	O5, team work
5. A reporting standard	Choose on validation, report test once, give the confidence interval, and use a McNemar test for "A beats B". This answers the weak points we found in the paper.	all RQs
6. Robustness argument	Show per-condition results (night, blur, fog) for single vs fused readers: the paper's real claim is fewer bad cases, which is what RQ4 asks.	RQ4

14. What benefit do you get from it?

- **A real, significant gain at almost no cost:** +1.84 points plate exact (12 fixed, 0 broken, p = 0.0005) for +0.8 ms per plate, and no new training.
- **A free error detector:** the layout check flags 10 % of readings, all of them wrong. In a live system you can reject them or wait for the next video frame; accepted readings are 94.9 % right.
- **A clear answer to "why not just ensemble everything?"** We can show with numbers (oracle, error overlap, calibration) when fusion helps and when it does not. Faculty like a negative result that is explained.
- **A methods contribution for the final report:** grammar-aware fusion for Bangla plates, with fair validation-based selection and significance tests, which the original paper lacks.
- **Direction for the next experiments:** the oracle says better voters are needed, the error breakdown says the char-YOLO decoder should enforce one letter and six digits, and the calibration plot says never to use HC.
- **A dataset fix for the whole team:** every teammate's OCR test contained the BLPD whole-photo rows. They cost our best reader 2.4 points and are now fixed in dataset v1.1.
- **Literature value:** a peer-reviewed reference (CIARP, LNCS) for both model fusion and multi-frame voting in the related-work section.

15. What to show the faculty (8 slides)

#	Slide title	What is on it	Source in this report
1	The question	single OCR models are uneven; can combining several help? how much? which?	section 1
2	The paper's method	12 models, HC / MV / MVCP, tie-breaks	Figure 1 A
3	The paper's results	92.4 → 97.6 % (8 sets), 87.6 → 90.3 % (unseen sets), HC worst, 4-6 fast models	Figure 2, section 5
4	Our critique	chosen on test; one run; per-dataset best already 96.8 %, so fusion adds 0.8 there	section 6
5	Our experiment	4 Bangla OCR models, same split, val-chosen; best single 85.56 %, fusion 86.18 % (n.s.)	Figure 3
6	Why plain fusion fails here	77 of 94 errors shared by all; CRNN overconfident	Figures 4 and 5
7	Our fix: BRTA-aware fallback	87.40 % (+1.84, 12 / 0, p = 0.0005) at 12.2 ms; best at night	Figure 1 B, Figures 6 and 8, example table
8	What it means for our system	final OCR, video voting, decoder fixes, dataset fix, team ensemble	sections 13 and 18

> **Keep it honest:** say "on our test split" for every number of ours, say that two of our four models are undertrained, and say that all choices were made on validation. Do not compare 86 % with the paper's 97.6 % as if it were the same test.

16. What to explain: a 3-minute script

"This paper, from the Brazilian group that also wrote our paper #23 on near-duplicates, asks a simple question: if we read the same plate with several different OCR models and combine the answers, do we read better? They trained 12 well-known models on 8 public datasets and combined them with three rules: trust the most confident model, take the majority string, or vote character by character. The majority string wins. The mean accuracy goes from 92.4 to 97.6 percent, and on unseen datasets from 87.6 to 90.3. Trusting the most confident model is the worst rule, because models are often confidently wrong. And four to six fast models give most of the gain.

When we read the tables closely, we found two weak points. They picked the best models on test results, and part of the gain only replaces knowing which model is best for each dataset: per dataset, the best single model already averages 96.8 percent.

We repeated the idea on Bangla plates with four different models trained on our merged dataset, and we chose everything on validation. Fusion gave only 0.6 points, which a significance test shows is noise. The reason is measurable: on 77 of the 94 plates our best model gets wrong, every other model is wrong too. Fusion cannot invent an answer that no model has. We also saw the paper's confidence problem: CRNN gives 60 percent of its wrong answers a confidence above 0.9.

Then we used something the paper does not have: the Bangladeshi plate format. A reading must have a district, one letter and six digits. Every reading that breaks this rule is wrong. So we keep our character-YOLO reading when it is valid, and ask CRNN only for the 10 percent that are invalid. That gives 87.4 percent, plus 1.8 points, with 12 plates fixed and none broken, which is significant, and it costs less than one millisecond per plate on average. It helps most at night.

For our project this means three things: the final reader is character-YOLO plus a format-aware fallback; the same format filter and majority vote will combine readings over video frames; and to gain more, we need better decoding of missed characters and stronger, more different second readers."

17. Questions the faculty may ask

Q1. The paper gets +5 points from fusion. Why do you get only 0.6?
Fusion needs several strong models that make different mistakes. The paper had 12 models between 86 and 92 %. We have one strong model, one medium and two weak ones, and their errors overlap: the oracle (any model right) is only 88.2 %, so no voting rule can give more than +2.6. Also, part of the paper's +5 is against one fixed model; against the best model per dataset the gain is 0.8.

Q2. Isn't +1.8 points also noise?
No. On the same 651 plates the fallback fixes 12 and breaks 0. The exact McNemar test gives $p = 0.0005$. The paper's rule (7 fixed, 3 broken) gives $p = 0.34$.

Q3. Did you tune on the test set?
No. The best model, the tie-break model, the rule and the subset were all chosen on the 766 validation plates; the test numbers were read once. This is stricter than the paper, which ranked models on test results.

Q4. Is the format check cheating? It knows what plates look like.
It uses only the public BRTA format, never the label of the plate. It is the same knowledge a human reader has. For a plate outside the format (army, diplomatic, old plates), char-YOLO's reading breaks the layout anyway; the rule then keeps it unless CRNN produces a valid BRTA string. Such plates are not in our test set, so this case is not measured yet.

Q5. Why are ViTSTR and the slot CNN so weak?
They were trained for about 5,000-9,000 steps on 5,600 crops, against 300,000 iterations on tens of thousands of images in the paper. ViTSTR starts to overfit after epoch 20. We did not tune them because the question was fusion; the next step is to train them longer with synthetic plates, or replace them with PARSeq from paper #31.

Q6. Why not calibrate the confidences and use HC?
The paper tried normalising confidences and it did not help. Temperature scaling on validation is a cheap test we can add, but MV-BM plus the format check already avoids the problem.

Q7. How does this help with video?
A vehicle is seen in many frames. The same rule combines the frames: drop invalid readings, take the majority string, and stop reading once three valid readings agree. That saves time and is our RQ3 and O4.

Q8. Do you reach the 90 % target?
Character accuracy is 95.9 %, above the target. Whole-plate accuracy is 87.4 %, and 89.9 % on real crops after removing the 19 BLPD whole photos.

Q9. Could all four models simply be bad on hard plates?
Yes, partly: on blurry plates, fog and rain every model drops (char-YOLO 79.5 % blurry, 73.1 % fog, 70.5 % rain). Fusion cannot fix what no model reads; better data (synthetic weather, super-resolution, paper #20) is the way there.

18. Next steps (in order)

1. **Dataset v1.1 (built 20 Oct, verify OK):** it has the BLPD crop fix and the paper #23 fair-list fix. **Your part:** upload `BanglaLPR_AllInOne_v1_1.zip` to the "All In One" folder (keep v1.0 there). Then put the new file id into `DRIVE_ZIP_ID` in `docs/_kit/lprkit/common.py` (the line is marked TODO) and run `build_kits.py`.
2. **Re-run paper #24 on v1.1** with the updated kit. The kit's fusion table now includes MV-BM+format, the fallback and a McNemar test for every row (added 20 Oct; on the 20 Oct predictions it reproduces 87.40 %, 12 fixed / 0 broken).
3. **Decoder fix in char-YOLO (O3):** keep one letter (the most confident), keep six digits (merge overlapping boxes), retry with a lower box threshold when a district or letter is missing. This targets the 64 invalid readings directly.
4. **Better second readers:** train ViTSTR and the slot CNN longer with synthetic plates, or use PARSeq (#31); re-check the oracle. Fusion only pays when it rises.
5. **Seeds:** 3 seeds for char-YOLO and CRNN, so the final table has mean ± spread.
6. **Video:** apply the format filter + majority vote over tracked frames and measure video-level accuracy and FPS (RQ3).

19. Files: GitHub and Drive

- I moved `results.zip` (298 MB), `paper24.log` and `paper24.ipynb` from the paper #2 folder, where they had been saved by mistake, to `docs/24_.../After Train/`.
- `results.zip` holds `paper24_outputs.zip` (124 MB, with weights; goes to Drive `498R / weights / paper24 /`) and `paper24_results.zip` (1.2 MB, no weights; goes to GitHub `weights/paper24/MD.Rokib.Hasan.Oli/` with `results_vs_paper.md`).
- Small weights for GitHub (web upload limit 25 MB): char-YOLOv8n best.pt 6.9 MB, slot CNN 6.9 MB, ViTSTR 21.7 MB. CRNN (30.8 MB) goes to Drive.
- Team table: put `paper24_outputs.zip` in `docs/results/` and run `python add_results.py paper24_outputs.zip --member "Rokib Hasan Oli"`; it fills the notes and results_table.md.
- Rebuild this report from `report_src/` (`analysis.py` → `charts.py` → `build_report.py`). It reads the v1.0 dataset zip, the version used for this run. v1.1 changes: `docs/DATASET_v1_1_CHANGES.md`.

Appendix: where every number comes from

Number	Source
Paper numbers	arXiv 2309.04331v2: Table 3 (p.9), Table 4 (p.9), Table 5 (p.11), Table 6 (p.12), text p.2-12
Per-dataset-best 96.8 % / 87.6 %	our arithmetic on Table 3 and Table 5 (best value per column, then the mean)
Single models, fusion table, oracle	Kaggle run 20 Oct 2026: results.zip → runs/*/results.json, runs/fusion/results.json
Overlap, McNemar, calibration, extra rules, fallback, groups	analysis.py on runs/*/ocr_preds_{val,test}.csv → analysis.json
Conditions, training-label counts	ocr/test_conditions.csv and ocr/train.csv inside BanglaLPR_AllInOne_v1.zip (v1.0)
BLPD whole photos	visual check of all 30 BLPD test images on 20 Oct 2026 (list in analysis.py)